

Outsourced projects

Rather than building systems by using their own staff, many organizations outsource their development efforts to contract development companies. They might outsource the work to take advantage of development skills they do not have available in-house, to augment their internal staff resources, to save money, or to accelerate development. The outsourced development supplier could be located physically nearby, on the other side of the world, or anywhere in between. Outsourced teams in other countries are typically referred to as being *offshore*. Offshoring is sometimes called *nearshoring* if the supplier's country is close by or shares a language and/or culture with the acquirer's country.

All outsourced projects involve distributed teams, with people working in two or more locations. The role of a business analyst is even more important on these projects than on a co-located project. Often, the BA's job is harder. If the team members are all in one location, developers can walk down the hall to ask the BA a question or to demonstrate newly developed functionality. This close collaboration can't happen in the same way with outsourced development, although modern communication tools certainly help. Compared to in-house development, outsourced—and particularly offshore—projects face requirements-related challenges such as the following:

- It's harder to get developer input on requirements and to pass along user feedback on delivered software to developers.
- A formal contractual definition of requirements is necessary, which can lead to contention if differences of interpretation are discovered late in the project.
- There might be a bigger gap between what the customers ultimately need and the product they get based on the initial requirements, because there are fewer opportunities to adjust the project's direction along the way.
- It might take longer to resolve requirements issues because of large time zone differences.
- Communicating the requirements is more difficult because of language and cultural barriers.
- Limited written requirements that might be adequate for in-house projects are insufficient for outsourced projects, because users and BAs are not readily available to answer developer questions, clarify ambiguities, and close gaps.
- Remote developers lack the organizational and business knowledge that in-house developers acquire with experience.

Although the original arguments for offshoring included anticipated cost savings based on hourly staff costs, many offshore projects actually experience a net *increase* in cost. Contributing factors include the additional effort required for more precise requirements, likely additional development iterations to close gaps because of unstated implied and assumed requirements, the additional overhead of the contractual arrangements, initial costs in developing effective norms of team behavior between the groups, and the costs of increased project communications and oversight throughout.

Software development work is the most common type of activity that is outsourced, but testing can also be outsourced. Outsourced testing presents the same challenges as outsourced development. Both types of activities rely on a solid foundation of clear requirements for success.

This chapter suggests techniques that are most important to enable successful requirements development and management on outsourced projects. This chapter does not discuss the decision process that leads to outsourcing the development or the process to select a vendor for the work.

Appropriate levels of requirements detail

Outsourcing product development to a separate company demands high-quality written requirements, because your direct interactions with the development team are likely to be minimal. As shown in Figure 23-1, you'll be sending the supplier a request for proposal (RFP), a requirements specification, and product acceptance criteria. Early on, both parties will engage in a review and will reach agreement, perhaps with negotiation and adjustments, before the supplier initiates development. The supplier will deliver the finished software product and supporting documentation.

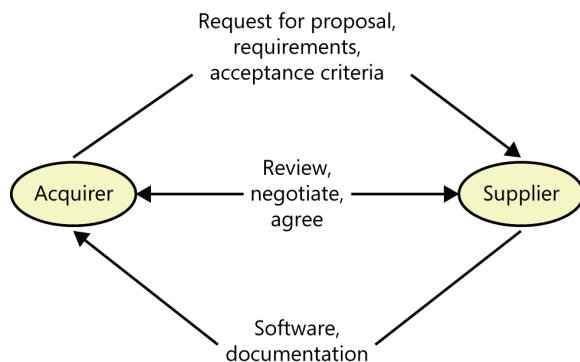


FIGURE 23-1 Requirements are the cornerstone of an outsourced project.

With outsourcing, you won't have the opportunities for day-to-day clarifications, decision making, and changes that you enjoy when developers and customers work in close proximity. Particularly with offshore development, you should anticipate that the supplier will build exactly what you ask them to build. You will get no more and no less, sometimes with no questions asked. The supplier won't implement the implicit and assumed requirements you thought were too obvious to write down. As a result, poorly defined and managed requirements are a common cause of outsourced project failure.

If you distribute an RFP, suppliers need to know exactly what you're requesting before they can produce realistic responses and estimates (Porter-Roth 2002). Because of the information that has to go into the RFP, you might have to develop more detailed requirements earlier in the project than on in-house development projects (Morgan 2009). At a minimum, specify a rich set of user requirements and nonfunctional requirements for the RFP. After the project is under way, you will likely need to specify all of the requirements with more precision than if an in-house team were building the same system, particularly if the outsourced team is offshore. If you are ever inclined to err on the side of overspecifying requirements, outsourced projects are the place to do so. It's the requirements author's responsibility to express the acquirer's expectations clearly. If certain deliverables must be produced for the acquirer to maintain a process certification or for compliance reasons, be sure to include those particulars as part of the RFP as well.

As with in-house development, visual requirements models augment functional and nonfunctional requirements for outsourced teams. Creating multiple representations of requirements increases the bandwidth of communication, so you might find it beneficial to create more models than if an in-house team were developing the software. Using representations like visual models to supplement written specifications is even more valuable if you are working across cultures and native languages, because it gives developers something to check their interpretations against. However, be sure the developers can understand the models you send them. If they aren't familiar with the models, that only raises the potential for confusion. One development manager was concerned that a written requirements specification plus mock-ups would not provide enough information for his offshore team to correctly implement a complex user interface (Beatty and Chen 2012). The display-action-response model described in Chapter 19, "Beyond requirements development," was developed specifically to meet the needs of this outsourced project.



Prototypes can also help clarify expectations for the supplier team. Similarly, the supplier can create prototypes to demonstrate to the acquirer their interpretation of the requirements and how they plan to respond to them. This is a way to create more customer-development interaction points to make course adjustments early in the project rather than late. Chapter 15, "Risk reduction through prototyping," has more information about creating and using prototypes.

Watch out for the ambiguous terms from Table 11-2 in Chapter 11, "Writing excellent requirements," that cause so much confusion. I once read an SRS intended for outsourcing that contained the word "support" in many places. The business analyst who wrote the SRS acknowledged that a contractor who was going to implement the software wouldn't know just what "support" meant in each case. A glossary is valuable when dealing with people who don't share the tacit knowledge held by those who are familiar with the acquiring company's environment. The structured keyword notation called Planguage (see Chapter 14, "Beyond functionality") can be used to describe requirements very explicitly for outsourced development (Gilb 2007).



Acquirer-supplier interactions

In the absence of real-time, face-to-face communication, you need other mechanisms to stay on top of what the supplier is doing, so arrange formal touch points between the acquirer and the supplier. In some outsourced projects, the supplier helps to write the functional requirements (Morgan 2009). This increases the initial costs associated with the outsourcing, but it also reduces the risk of misunderstandings.

Plan time for multiple review cycles of the requirements. Use collaboration tools to facilitate peer reviews with participants in multiple locations (Wiegers 2002). Be aware, though, that members of certain cultures find it difficult to offer even constructive criticism of another person's work. Authors in such a culture whose work is being reviewed could take review comments personally (Van Veenendaal 1999). The result is that the reviewers might sit politely during the peer review, saying nothing because they don't want to offend the author. This is courteous and considerate, but it does not contribute to a shared goal of discovering requirements defects as early as possible to make development cheaper and faster. Discover whether this cultural characteristic applies to your outsource partners so you can determine realistic expectations and strategies for your peer reviews.



The project schedule for one failed offshore project included a one-week task named "Hold requirements workshops," followed immediately by tasks to implement several subsystems (Wiegers 2003). The supplier forgot to include vital intermediate tasks to document, review, and revise the requirements specifications. The iterative and communication-intensive nature of requirements development dictates that you must allow sufficient time for these review cycles. The acquirer and the supplier on this project were in different countries, at opposite ends of the same continent. They experienced slow turnaround on the myriad questions that arose as the SRS cycled back and forth. Failure to resolve requirements issues in a timely way derailed the schedule and contributed to eventually sending the two parties into litigation.

Peer reviews and prototypes provide insight into how the supplier is interpreting the requirements. Incremental development is another risk-management technique that permits course corrections when a misunderstanding sends the supplier's developers in the wrong direction. If the supplier raises questions, document them and integrate the answers into the requirements (Gilb 2007). Monitor the resolution of the questions in an issue-tracking tool to which both supplier and acquirer teams have access, as described in Chapter 27, "Requirements management practices."

Contract development companies that work on many types of projects might lack the specific domain or company knowledge that is critical to making the right decisions. Consider delivering some training to the contractor staff about the project and application domain prior to requirements review, to try to bridge this knowledge gap.

Outsourced projects often involve teams with disparate company cultures and attitudes. Some suppliers will be so eager to please that they agree to outcomes they cannot deliver. When an error is brought to their attention, they might strive to save face by not fully accepting responsibility for the problems. Additional cultural differences arise with offshore suppliers. Some developers might hesitate to ask for help or clarification. They might be reluctant to say "no" or "I don't understand." This can lead to misinterpretations, unresolved issues, and unachievable commitments. To avoid these

issues, employ elicitation and facilitation techniques such as reading between the lines for what isn't said and asking open-ended questions to gain accurate visibility into issues and status. Consider establishing ground rules with your team members, both local and remote, to expressly define how the team members should interact when they work together.

Developers whose first language is different than the language in which the requirements are written are likely to interpret requirements literally, not picking up nuances or fully appreciating the implications. They might make user interface design choices that you wouldn't expect. Things as diverse as date formats, systems of measurement (such as United States customary units, SI units, or imperial units), the symbolism of colors, and the order of people's given and family names can vary between countries. When interacting with people who have a different native language from yours, make your intentions and desires as clear as possible in simple language. Avoid the use of colloquialisms, jargon, idioms, and references to pop culture that could be misconstrued.



One offshore team took a customer's requirements very literally. It was as though the developers translated each requirement from English into their own language, coded it, moved on to the next requirement, and continued until they reached the end of the list. The product that was delivered to the customer technically met the requirements, but it fell far short of meeting expectations. The developers weren't trying to be difficult. They just didn't understand the language of the requirements very well. Consequently, they never fully grasped the essence of what they were building. The customer brought most of the development work back in-house and effectively had to pay twice to have the software developed correctly.

Trap Don't assume that suppliers will interpret ambiguous and incomplete requirements the same way that you do. The burden is on the acquirer to communicate all necessary information to the supplier, using frequent conversations to resolve requirements questions. But the burden is on the supplier to proactively ask clarifying questions instead of making assumptions that could be incorrect.

Change management

At the beginning of the project, establish a mutually acceptable change control process that all participants can use, no matter where they're located. Using a common set of web-based tools for handling change requests and tracking open issues is essential. Change always has a price, so using change management practices to control scope creep is vital in a contract-development situation. Identify the decision makers for proposed changes and the communication mechanisms you'll use to make sure the right people are kept informed. Most outsourced work has contractual agreements in place to describe exactly what the development team must deliver. The contract should specify who will pay for various kinds of changes, such as newly requested functionality or corrections made in the original requirements, and the process for incorporating the changes into the product. When there is misalignment between requirements and delivery, the arguments that ensue are consequently also contractual in nature. Unfortunately, often both parties lose (McConnell 1997).

Acceptance criteria

In keeping with Stephen Covey's recommendation to "begin with the end in mind" (Covey 2004), define in advance how you'll assess whether the contracted product is acceptable to you and your customers. How will you judge whether to make the final payment to the supplier? If the acceptance criteria are not fully satisfied, who is responsible for making corrections, and who pays for those? Include acceptance criteria in the RFP so the supplier knows up front what to expect. Validate the requirements before you give them to the outsourced team, to help ensure that the delivered product will be on target. Chapter 17, "Validating the requirements," suggested some approaches to defining acceptance criteria, as well as methods for reviewing and testing requirements.

Properly handled, outsourcing the development work can be an effective strategy to build your software system. Building collaborative relationships with outsourced development suppliers is challenging because of distance, language and cultural differences, and potentially competing interests. Suppliers might not be motivated to correct any requirement errors or ambiguities discovered along the way if they will be paid more to fix the problems following delivery of a release candidate. An essential starting point on a journey to a successful outsourced development experience is a set of high-quality, complete, and explicitly clear requirements. If the requirements you provide to the supplier are incomplete or misunderstood, failure of the project is probably at least as much your fault as theirs.